

Bandsaw Monitor System Quick Start

User & Operator Guide

MATRIX Project

Version 1.0 - May 04, 2026

Table of Contents

1	Overview	3
1.1	What is the Bandsaw Monitor System?	3
1.2	System Components	3
2	Getting Started	6
2.1	System Requirements	6
2.2	Installation	6
2.3	Initial Setup	6
3	Core Features	8
3.1	Video Capture	8
3.2	Hand Detection	9
3.3	Slit Detection	9
3.4	Alert System	10
4	Using the Dashboard	12
4.1	Launching the Application	12
4.2	Dashboard Interface	12
4.3	Common Tasks	12
5	Troubleshooting	12
5.1	Common Issues	12
6	Frequently Asked Questions	14
7	Support & Resources	15
7.1	Getting Help	15
7.2	Additional Resources	15
8	Appendix	16
8.1	A. Installation Troubleshooting	16
8.2	B. Hardware Setup Guide	16
8.3	C. Command Reference	16

1 Overview

1.1 What is the Bandsaw Monitor System?

The Bandsaw Monitor System is a real-time safety monitoring solution designed to protect operators during bandsaw operations. Using computer vision and AI-powered hand detection, the system continuously monitors for operator hands near the cutting blade. When a hand enters the danger zone while the blade is active, the system immediately triggers visual alerts to prevent injuries.

Key Features:

- Real-time hand detection using MediaPipe
- Blade and slit position tracking
- GUI dashboard for monitoring and configuration
- Future Containerized deployment through Docker

1.2 System Components

The Bandsaw Monitor System consists of:

- **Video Capture:** Real-time video processing from camera feeds
- **Hand Detection:** AI-powered hand presence detection for safety monitoring
- **Slit Detection:** Blade and workpiece position detection
- **Alerts:** Notification system for safety events
- **Monitoring Dashboard:** GUI for system status and monitoring

1.2.1 System Architecture Diagram

Bandsaw Monitor System Architecture, Chat GPT-generated diagram illustrating component flow and data paths

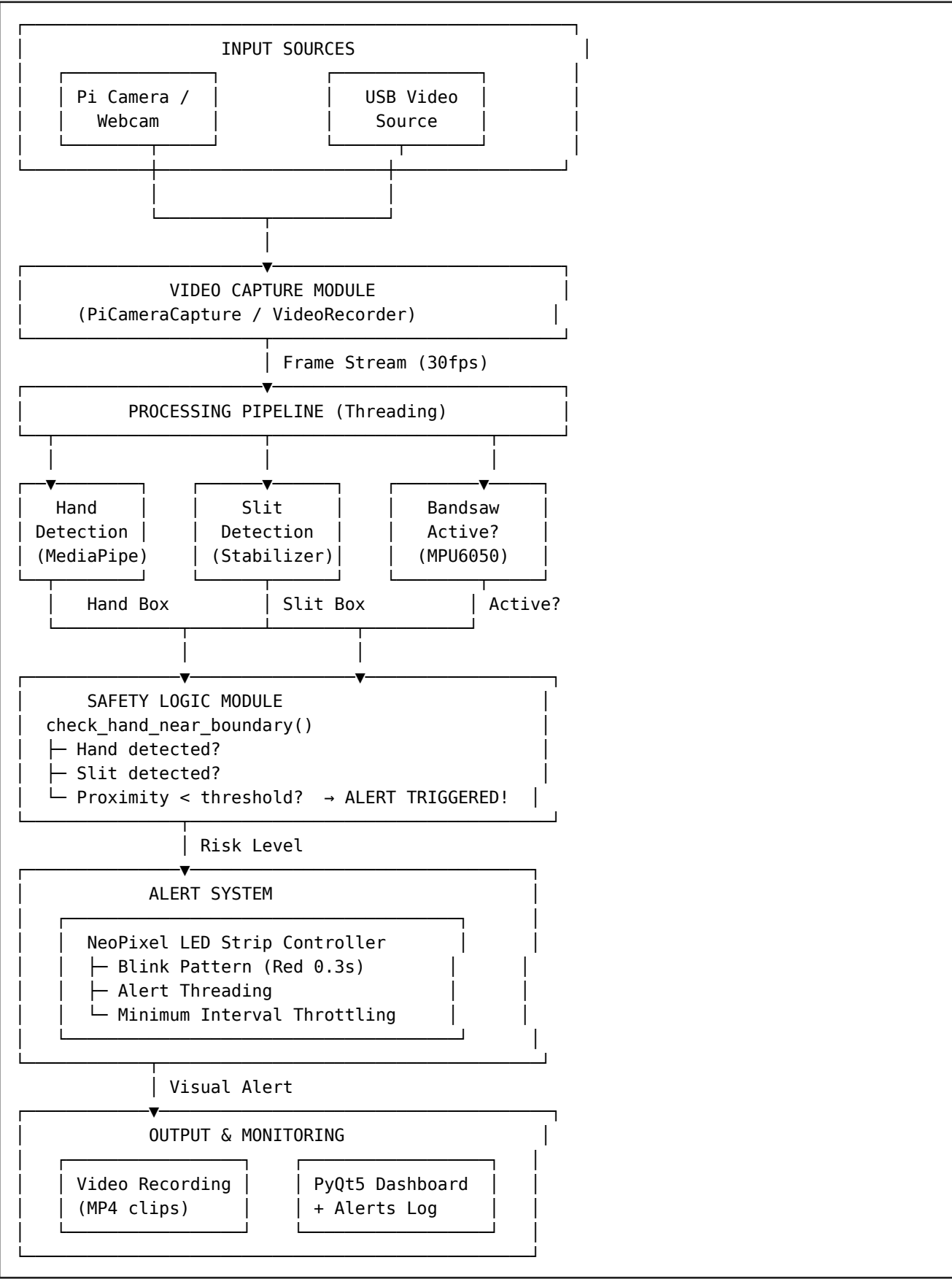


Table 1: Figure 1: System Architecture - Component Flow and Data Paths

2 Getting Started

2.1 System Requirements

Hardware Requirements:

- Camera: Raspberry Pi Camera Module
- Processor: Raspberry Pi 5

Software Requirements:

- Python 3.11 or higher
- Operating System: Raspberry Pi OS / Ubuntu Linux / Windows 10+
- Dependencies installed: numpy, opencv-python, mediapipe, smbus2, picamera2

2.1.1 Hardware Block Diagram

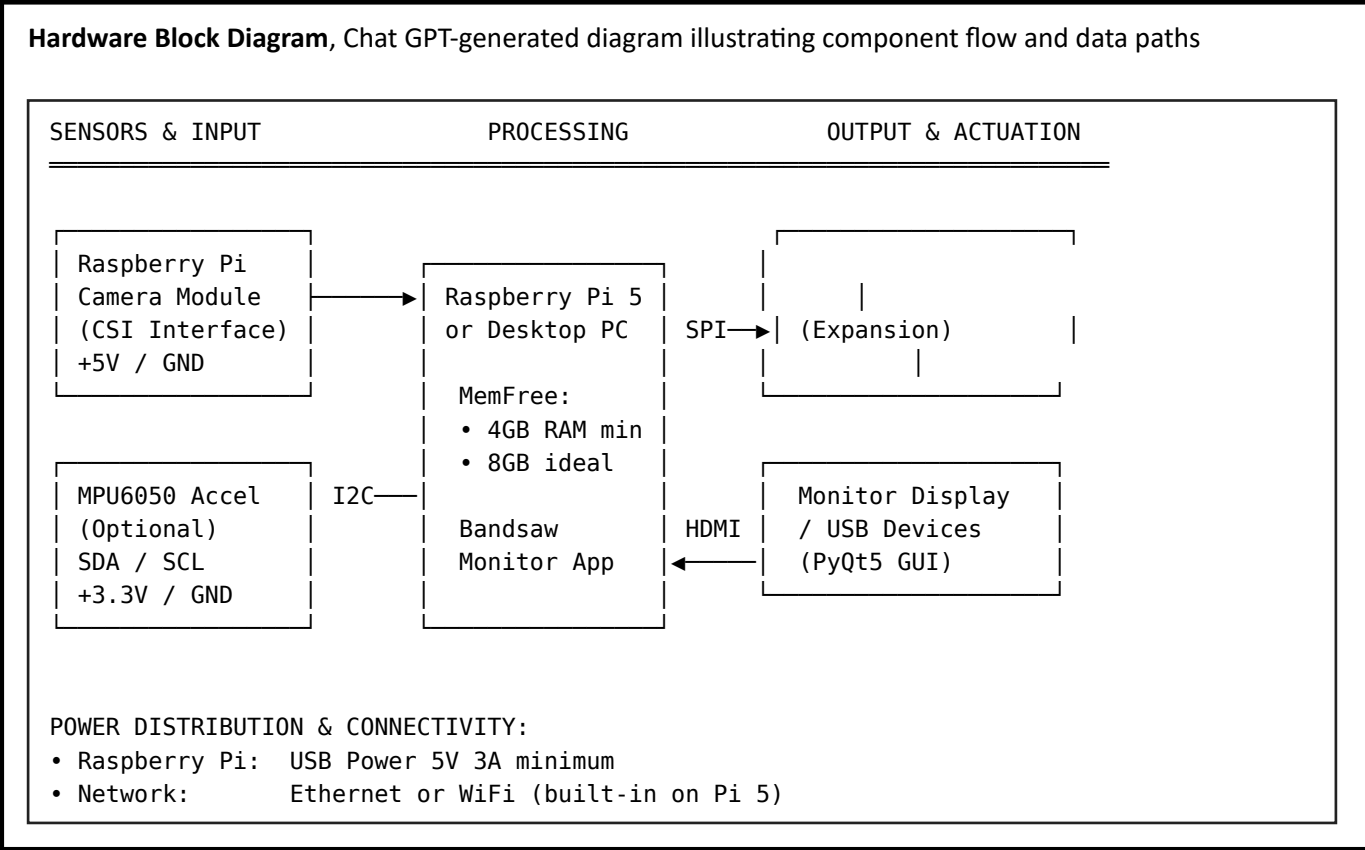


Table 2: Figure 2: Hardware Connections and Pin Layout

2.2 Installation

2.2.1 Option 1: From Source

```
git clone https://github.com/GeorgiaTech-DDI/bandsaw.git
cd bandsaw/01-bandsaw_software
pip install -e .
```

Additional Setup for Raspberry Pi:

```
# Install system dependencies
sudo apt-get update
sudo apt-get install -y python3-pil python3-numpy python3-pip
```

2.3 Initial Setup

1. **Hardware Connections:** Connect camera to CSI port (Raspberry Pi) or USB port (Desktop). Connect power.
2. **Verify Camera Detection:** Run `libcamera-hello` (Pi) or test with `v4l2-ctl` to confirm camera is detected and working.

3. **Validate Dependencies:** Run the test suite with `pytest tests/robot/` to ensure all modules load correctly.
4. **Test System:** Launch the monitoring system with `python -m bandsaw.bandsaw_monitor --debug` to verify hand detection and alerts are working.

3 Core Features

3.1 Video Capture

3.1.1 Overview

The video capture module handles real-time images from Raspberry Pi Camera.

3.1.2 Video Processing Pipeline Diagram

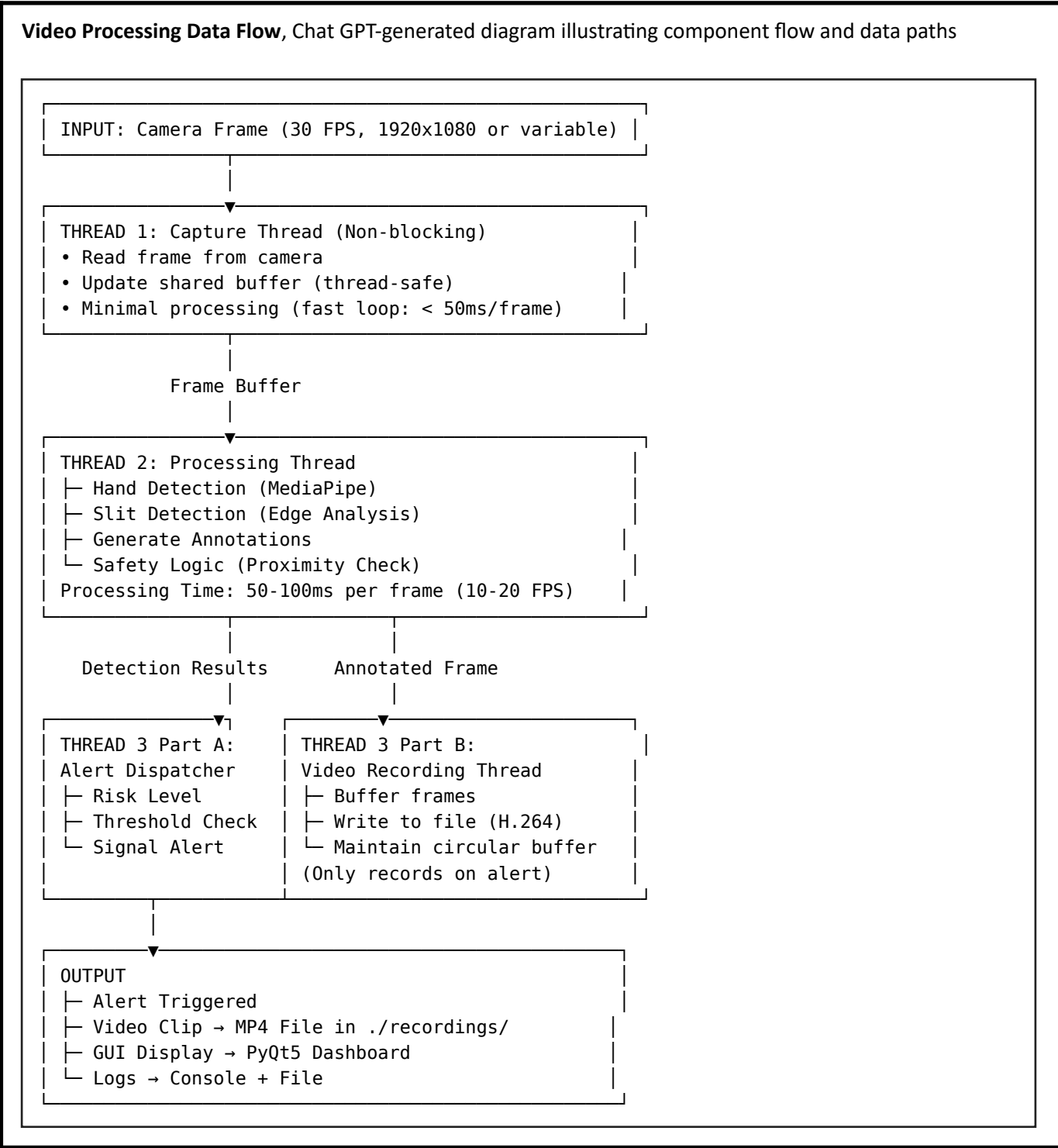


Table 3: Figure 3: Multi-threaded Video Processing Pipeline

3.1.3 Configuration

Configure via command-line arguments when launching the monitor:


```
python -m bandsaw.bandsaw_monitor \
  --video 0 \
```

3.1.4 Troubleshooting

Issue: “Cannot open camera”

- Verify camera is physically connected
- Check permissions: `sudo usermod -a -G video $USER`

—

3.2 Hand Detection

3.2.1 Overview

Hand detection uses **Google MediaPipe Hands**, a state-of-the-art real-time hand tracking model. The system can detect up to 2 hands simultaneously with 21 landmark points per hand, enabling precise bounding box calculation. The model uses machine learning to identify hands and their position regardless of orientation or size.

3.2.2 Configuration

Configuration is currently hardcoded in `hand_detection.py`:

```
hands_detector = mp.solutions.hands.Hands(
    static_image_mode=False,
    max_num_hands=2,
    min_detection_confidence=0.7,
    min_tracking_confidence=0.7
)
```

3.3 Slit Detection

3.3.1 Overview

Slit detection tracks the blade position and cutting boundaries during bandsaw operation. The system identifies where the blade intersects with the material and maintains a stabilized estimate of the active cutting area. This information is used to determine if detected hands are actually in the danger zone.

How It Works:

- Analyzes frame contrast to identify blade edges
- Tracks slit position across consecutive frames
- Applies temporal smoothing to reduce jitter
- Stores detected slit position over a configurable stabilization period
- Finalizes slit position once threshold is met

3.3.2 Configuration

Configure via command-line arguments:

```
python -m bandsaw.bandsaw_monitor \
  --stabilization 5.0 \
  --threshold 0.5
```

Configuration Parameters:

- `--stabilization SECONDS`: Time window for slit position stabilization (default: 5.0 seconds). Longer = more stable but slower to detect changes
- `--threshold VALUE`: Vibration/motion threshold (default: 0.5, currently unused)

Tuning Guidance:

- Lower stabilization time (2-3s) for faster response, less stable detection
- Higher stabilization time (7-10s) for more stable position tracking, slower response to blade movement
- Typical range: 3-5 seconds

3.3.3 Usage

The slit detection is automatically managed by the BandsawMonitor class:

```
from bandsaw.slit_detection import SlitDetector

slit_detector = SlitDetector(
    stabilization_time=5.0,
    debug=True # Enables console output
)

# Start detection when bandsaw begins operation
slit_detector.start_detection()

# In monitoring loop
slit_bbox = slit_detector.detect(frame)
# slit_bbox = (x, y, width, height, angle) or None

# Stop detection when bandsaw stops
slit_detector.stop_detection()
```

3.4 Alert System

3.4.1 Overview

The alert system provides immediate visual feedback when safety conditions are violated. When a hand is detected approaching or entering the slit (blade area), the system activates addressable NeoPixel LED strips to create a blinking alert pattern. This provides real-time feedback to both the operator and nearby personnel.

3.4.2 Alert Types

- **Hand Detected Near Slit:** Steady red blinking (0.3 second intervals) when hand bounding box overlaps with blade area
- **High-Risk Proximity:** Continuous red illumination for critical danger zone violations
- **System Clear:** Green when hand leaves danger zone or bandsaw becomes inactive

Alert Timing:

- Minimum alert interval: 2.0 seconds (prevents alert spam from multiple detections)
- Blink interval: 0.3 seconds (fast, attention-grabbing)
- Alert colors: Red (0xFF0000) for danger can be customized

3.4.3 Configuring Alerts

Configure via command-line arguments:

```
python -m bandsaw.bandsaw_monitor \
    --alert-interval 2.0
```

Configuration Parameters:

- `--alert-interval SECONDS`: Minimum time between consecutive alerts (default: 2.0 seconds)

Programmatic Configuration:

```
from bandsaw.alert import start_alert, stop_alert, clear_alert

# Trigger red alert
start_alert(color=(255, 0, 0)) # RGB tuple

# Stop alert (turn off LEDs)
stop_alert()

# Clear all LEDs
clear_alert()
```

Customization: Edit `alert.py` to change:

- `NUM_PIXELS`: Number of LEDs in your strip
- `brightness`: LED brightness (0.0-1.0)
- Color values in `start_alert()` calls

4 Using the Dashboard

4.1 Launching the Application

Option 1: Command-line Monitoring (Recommended for Raspberry Pi)

```
python -m bandsaw.bandsaw_monitor --debug
```

Option 2: PyQt5 GUI Dashboard

```
python gui/pyqt_app.py
```

The GUI provides a graphical interface with real-time video feed visualization and alert status.

Option 3: Headless Remote Monitoring

```
python gui/headless_monitor.py --host 0.0.0.0 --port 5000
```

4.2 Dashboard Interface

The PyQt5 dashboard provides the following components:

- **Status Panel:** Displays current system state (Running/Stopped), bandsaw active status, detected hands, alert count
- **Live Video Feed:** Real-time camera stream with hand detection bounding boxes and slit detection overlay
- **Alerts Log:** Chronological list of all detected hand-slit proximity events with timestamps
- **Settings Panel:** Adjustable parameters for detection sensitivity, alert thresholds, and recording options
- **Control Buttons:** Start/Stop monitoring, Clear alerts, Export video clips

4.3 Common Tasks

4.3.1 Start Monitoring

1. Launch the monitor: `python -m bandsaw.bandsaw_monitor`
2. Verify camera stream appears in terminal or GUI
3. System is now actively monitoring for hand detection

4.3.2 Stop Monitoring

1. Press `Ctrl+C` in terminal
2. Any active recordings will be saved to the recordings directory

4.3.3 View Alert History

1. Alerts are logged in real-time during monitoring
2. Check the alerts log file: `./logs/alerts.log`
3. In GUI: View the Alerts Log panel which shows all detections with timestamps
4. Each entry shows: timestamp, hand count, slit status, proximity level

4.3.4 Export Data

1. Video recordings are automatically saved to the recordings/ directory
2. Export format: MP4 video files named with timestamp
3. To manually export: Copy the recordings folder to external storage
4. Logs can be exported as CSV: `python -m bandsaw.utils.export_logs output.csv`

5 Troubleshooting

5.1 Common Issues

5.1.1 Issue: Camera Not Detected

Symptoms: “Error: No camera available” or black/frozen video feed

Solutions:

1. **For Raspberry Pi Camera:** Enable CSI camera in raspi-config:

```
sudo raspi-config
# Navigate to: Interface Options > Camera > Enable
# Reboot
```

2. **Verify camera connection:**

```
libcamera-hello --verbose # Pi OS
v4l2-ctl --list-devices   # Linux
```

3. **Check permissions:**

```
sudo usermod -a -G video $USER
# Log out and back in, or use: newgrp video
```

4. **Try alternative video source:**

```
# Test USB webcam
python -m bandsaw.bandsaw_monitor --video /dev/video0
```

5. **Debug camera initialization:**

```
python -m bandsaw.bandsaw_monitor --debug
# Look for camera initialization messages in output
```

5.1.2 Issue: Hand Detection Not Working

Symptoms: No bounding boxes appear around hands, or constant false detections

Solutions - No Detection:

1. **Verify lighting:** Hand detection requires adequate lighting. Test the camera feed visually first.

```
libcamera-hello # Or run GUI to see raw video
```

2. **Check MediaPipe installation:**

```
python -c "import mediapipe; print(mediapipe.__version__)"
# Should print version (e.g., 0.10.32)
```

3. **Lower detection sensitivity:**

- Edit bandsaw/hand_detection.py line 14
- Change min_detection_confidence=0.7 to 0.5 or 0.6
- More permissive but may increase false positives

4. **Debug hand detection:**

```
from bandsaw.hand_detection import detect_hands
import cv2
cap = cv2.VideoCapture(0)
ret, frame = cap.read()
hands, annotated = detect_hands(frame)
print(f"Detected {len(hands)} hands")
```

Solutions - Too Many False Detections:

1. **Increase confidence threshold:** Edit line 14, change to min_detection_confidence=0.85
2. **Improve lighting:** Better lighting reduces false detection in shadows
3. **Clean camera lens:** Smudged lens causes detection errors

5.1.3 Issue: Alerts Not Triggering

1. **Verify hand-slit proximity detection:**

- Run with `--debug` flag to see detection logs
- Confirm `check_hand_near_boundary()` returns `True`
- May need to adjust bounding box thresholds in `bandsaw_monitor.py`!!!

2. Check alert interval:

- Reduce interval: `--alert-interval 1.0`

—

5.1.4 Issue: System Performance is Slow

Symptoms: Low FPS (< 5 fps), laggy video, delayed detections (>500ms)

Solutions:

1. Check system resources:

```
top # Monitor CPU and memory usage
# Typical usage: 15-20% CPU, 100-200MB RAM
```

2. Reduce video resolution: Edit `bandsaw/video_capture.py`

- Lower resolution = faster processing
- Change resolution configuration from 1920x1080 to 1280x720 or 640x480

—

6 Frequently Asked Questions

Q: How often should I maintain the system?

A: Maintenance recommendations:

- **Monthly:** Clean camera lens with soft, dry cloth
- **Quarterly:** Check NeoPixel LED connections for corrosion
- **Annually:** Replace camera if dust buildup prevents focus
- **As needed:** Update software with `pip install --upgrade bandsaw`

—

Q: Can I customize the alerts?

A: Yes. Modify `bandsaw/alert.py` to:

- Change LED color: Edit RGB values in `start_alert()` call
- Change blink pattern: Modify interval in `_blink_loop()` function
- Change number of LEDs: Update `NUM_PIXELS` variable
- Add audio alerts: Integrate sound synthesis library in alert function

—

Q: What if the system detects false hands (objects, shadows)?

A: Several mitigation strategies:

1. Increase detection confidence threshold (0.8-0.85) to filter out uncertain detections
2. Improve lighting conditions near the bandsaw
3. Train a custom hand detection model on your specific lighting conditions
4. Implement a confidence-weighted alert system that requires multiple consecutive detections

7 Support & Resources

7.1 Getting Help

- **Issue Tracker:** <https://github.com/GeorgiaTech-DDI/bandsaw/issues>
- **GitHub Repository:** <https://github.com/GeorgiaTech-DDI/bandsaw>
- **MATRIX Lab VIP:** Georgia Tech DDI MatrixLab
- **Email Support:** rlohan@gatech.edu

7.2 Additional Resources

- **MediaPipe Documentation:** https://developers.google.com/mediapipe/solutions/vision/hand_landmarker
- **Raspberry Pi Camera Documentation:** <https://www.raspberrypi.com/documentation/cameras/>
- **NeoPixel LED Documentation:** <https://learn.adafruit.com/adafruit-neopixel-uberguide>
- **PyQt5 Documentation:** <https://www.riverbankcomputing.com/static/Docs/PyQt5/>
- **Project Repository:** <https://github.com/GeorgiaTech-DDI/bandsaw>

Create an Issue: Visit: <https://github.com/GeorgiaTech-DDI/bandsaw/issues/new>

8 Appendix

8.1 A. Installation Troubleshooting

Error: “ModuleNotFoundError: No module named ‘mediapipe’”

Solution:

```
pip install mediapipe
# For Pi with arm64: May require compilation
pip install mediapipe --no-binary mediapipe
```

Error: “ModuleNotFoundError: No module named ‘picamera2’”

Solution (Raspberry Pi only):

```
sudo apt-get install -y python3-picamera2
```

Error: “OSError: picamera2 not found”

Solution: Not on Raspberry Pi hardware. Fall back to OpenCV WebCam:

```
# Edit bandsaw/bandsaw_monitor.py
# Comment out: self.camera = PiCameraCapture()
# Use: self.cap = cv2.VideoCapture(0) # Standard webcam
```

8.2 B. Hardware Setup Guide

Raspberry Pi Camera Setup:

1. Power off the Raspberry Pi
2. Open camera slot on Pi board
3. Insert ribbon cable (blue contacts face outward)
4. Close and clamp the slot
5. Boot Pi and run: libcamera-hello

8.3 C. Command Reference

Launch Monitoring System:

```
python -m bandsaw.bandsaw_monitor [OPTIONS]
```

Full Option List:

```
--debug          Enable debug logging
--video SOURCE    Video source (default: 0, or filepath)
--record DIRECTORY Recording output directory (default: recordings/)
--threshold VALUE Vibration threshold (default: 0.5, reserved)
--stabilization SEC Slit stabilization time (default: 5.0)
--alert-interval SEC Alert minimum interval (default: 2.0)
```

Launch GUI Dashboard:

```
python gui/pyqt_app.py
```

Launch Headless Monitor (Web API):

```
python gui/headless_monitor.py --host 0.0.0.0 --port 5000
# Access at: http://<pi-ip>:5000
```

Run Tests:

```
pytest tests/
pytest --cov=bandsaw tests/
```